1 Input

1.1 getline

1. Not part of POSIX standard!
2. Reads newline terminated string.
3. Allocates memory for us, will resize the buffer internally if too small.
4. No overflows!
5. Returns -1 on error, string length on success.

```
char *buffer = NULL;
size_t buf_size = 0;
ssize_t str_len = getline(&buffer, &buf_size, stdin);
if(str_len == -1) ERR("getline");
buffer[str_len - 1] = '\0'; // Optionally insert null char in place of newline char at
    the end
// ...
free(buffer);
```

1.2 fgets

1. Reads new line terminated string.
2. No overflows... but if buffer is not big enough will read only up to its size.
3. Returns NULL on error, buffer pointer on success.

```
#define BUF_SIZE 256
    //...
    char buffer[BUF_SIZE];
    if(NULL == fgets(buffer, BUF_SIZE, stdin))
        ERR("fgets");
    buffer[strlen(buffer) - 1] = '\0'; // Optionally insert null char in place of newline
        char at the end (assuming string fits in the buffer)
```

1.3 fgetc

1. Reads a single character.
2. Returns EOF on error, character on success.
3. There might still be other character in the stdin...
4. If the user input was "ABC", there are still 'B', 'C' and '\n' chars in the stdin you might want to read. Consider reading the whole line and taking one character from it.

```
int c = fgetc(stdin);
if(c == EOF) ERR("fgetc");
```

1.4 scanf

1. Dangerous if used improperly.

2 Output

2.1 puts

1. Outputs a string to stdout and a trailing newline character.
2. Use it when there is no string formatting.
3. Returns EOF on error.

```
char *str = "Hello!";
int ret = puts(str);
if(ret == EOF) ERR("puts");
```

2.2 fputs

1. Outputs a string without adding a trailing newline character.
2. Use it when there is no string formatting.
3. Returns EOF on error.

```
char *str = "Hello!";
int ret = fputs(str, stdout);
if(ret == EOF) ERR("fputs");
```

2.3 putc

1. Outputs a single character.
2. Returns EOF on error.
3. See also: fputc, putchar.

```
char c = 'H';
int ret = putc(c, stdout);
if(ret == EOF) ERR("putc");
```

2.4 printf

1. Outputs a formatted string to stdout.
2. Use when there is some string formatting involved.
3. Never print user inputted text using printf!
4. Returns the number of characters printed on success, negative value on failure.
5. See also: fprintf.

```
char *str = "Hello!";
int n = 4;
/* print at most first four characters */
int ret = printf("%.*s\n", n, str);
if(ret < 0) ERR("printf");
```